



Mushroom Classification Analyzer

Un progetto di Machine Learning per prevedere se un fungo è **commestibile** o **velenoso**, utilizzando Logistic Regression e XGBoost.

Disponibile su: <https://github.com/gianlucameni/logistic-regression-and-XGBoost>

A cura di: Gianluca Meni e Samuele Ghezzi

Indice dei Contenuti

1. Dataset Mushroom

Esplorazione e caratteristiche del dataset

2. Struttura del Progetto

Architettura e componenti

3. Gestione dei Dati e Encoding

4. Correlazione con il Target

5. Metriche di Valutazione

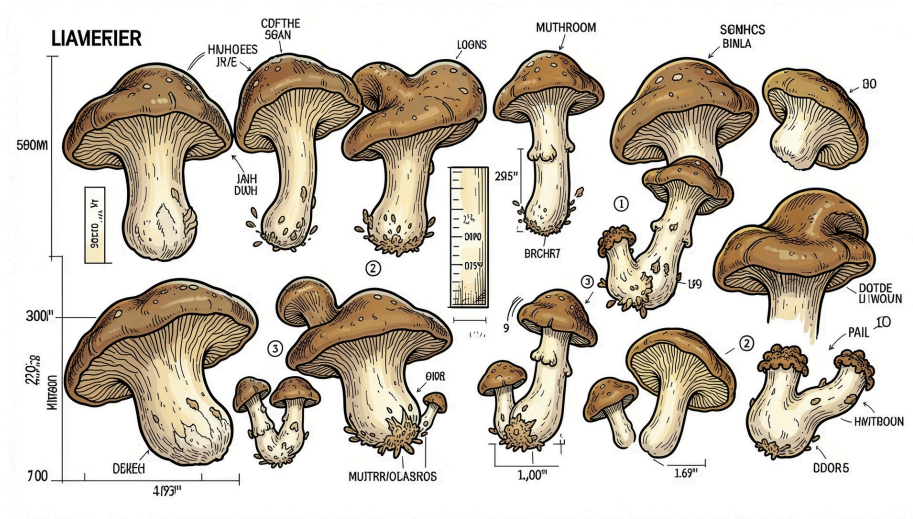
6. Regressione Logistica

7. XGBoost

8. Risultati e Confronto Modelli

9. Sviluppi Futuri

Il Dataset Mushroom



Fonte

Dataset [UCI Mushroom](https://archive.ics.uci.edu/dataset/8/mushroom), per la classificazione binaria.

Caratteristiche

- **8.124 campioni** di funghi
- **22 feature** categoriche (colore, forma, odore, habitat,...)
- **Target binario:** Edible (e) / Poisonous (p)

Struttura del Progetto

main.py

Script principale per avviare il flusso di analisi, training e valutazione.

test.py

Script dedicato ai test e alla verifica del comportamento dei componenti del progetto.

Dockerfile

Definisce l'ambiente Docker per rendere l'esecuzione del progetto riproducibile.

requirements.txt

Elenca tutte le dipendenze Python necessarie per installare ed eseguire il progetto.

README.md

Fornisce una panoramica del progetto, istruzioni di setup e modalità d'uso.

it/valtellina

Package principale che raccoglie i moduli dell'applicazione.

analyzer

Modulo per l'analisi e la preparazione dei dati.

graphs

Modulo per la creazione di grafici.

logistic_regression

Contiene la definizione, il fitting e la valutazione del modello di Logistic Regression.

xgboost

Contiene la definizione, il fitting e la valutazione del modello XGBoost.

one_hot_encoder

Gestisce la codifica delle feature categoriche tramite one-hot encoding.

splitter_train_test

Si occupa della suddivisione del dataset in training set e test set.

Gestione dei Dati e Encoding

Data Handling

I valori mancanti sono stati identificati e gestiti introducendo un nuovo valore "Unknown", in quanto presenti 2480 nella colonna "stalk-root".

La colonna "veil-type" presenta un solo valore costante, per cui è stata eliminata.

Split Train/Test

Il dataset è stato suddiviso in **training set** e **test set** prima di qualsiasi trasformazione.

Encoding delle Variabili

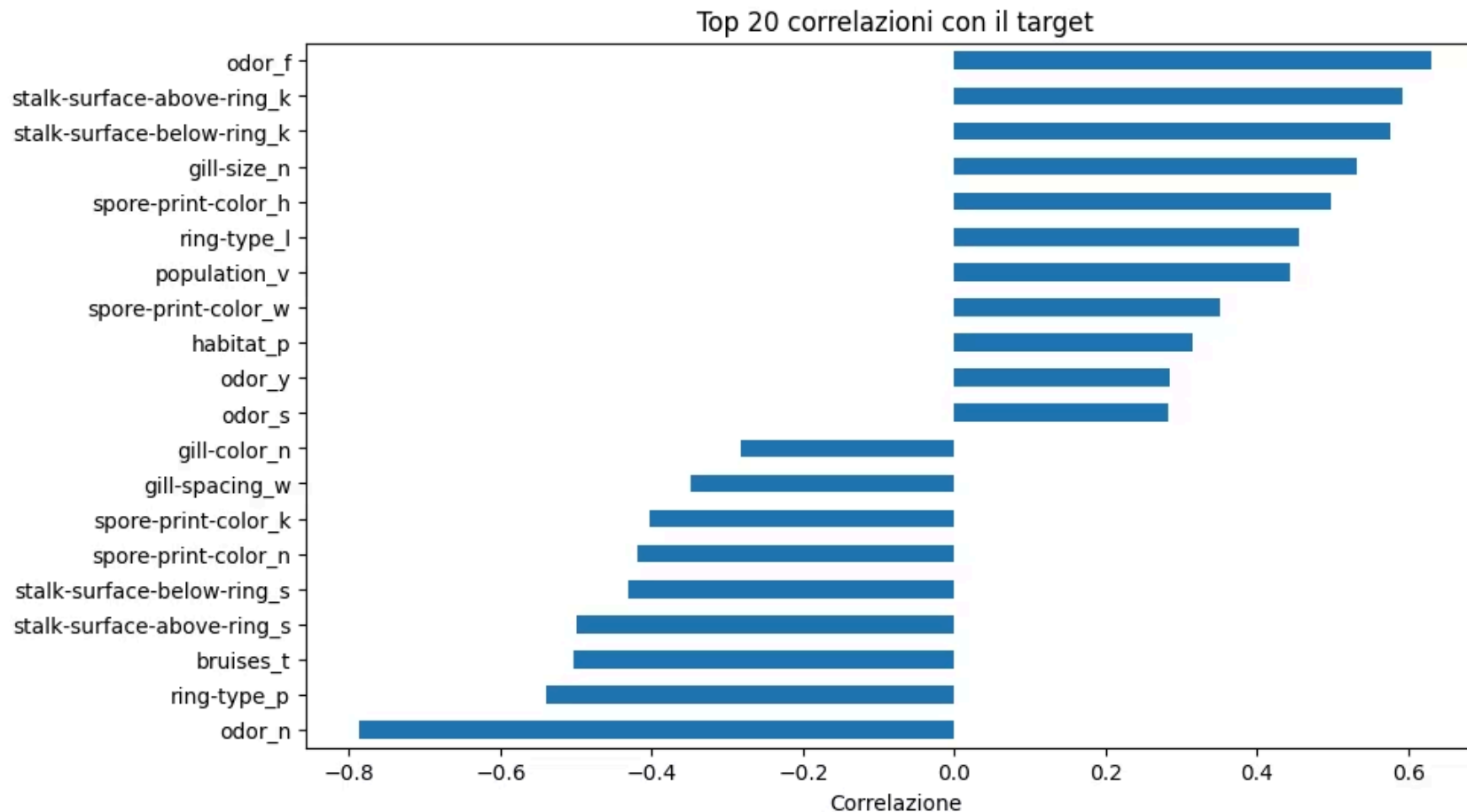
One-Hot Encoding

Applicato a tutte le **22 feature categoriche** per convertirle in formato numerico compatibile con i modelli.

Label Encoding

Il target è stato codificato come **0 (Edible)** e **1 (Poisonous)** per la classificazione binaria.

Correlazione con il Target



Top 20 feature

Le feature più correlate con la commestibilità.

odor

Feature più rilevante, positivamente con **odor_f** e negativamente con **odor_n**.

Correlazioni positive

Associate ai funghi velenosi.

Correlazioni negative

Associate ai funghi commestibili.

Metriche di Valutazione

Le performance dei due modelli sono state valutate sul test set con le metriche standard per la classificazione binaria.



Accuracy

Percentuale di previsioni corrette sul totale. Indica la bontà complessiva del modello.



Precision

Quanti dei predetti velenosi lo erano davvero. Importante per ridurre i falsi allarmi.



Recall

Quanti velenosi reali sono stati identificati su tutti i velenosi presenti.



F1-Score

Media armonica di Precision e Recall. Bilancia i due aspetti in un unico indicatore.

Regressione Logistica

- Modello per **classificazione binaria**
- Restituisce una **probabilità di appartenenza alla classe positiva (velenoso)**
- Usa la funzione **sigmoide** per trasformare l'output in una probabilità compresa **tra 0 e 1**
- Obiettivo: **predire probabilità in modo interpretabile**

Accuracy

99.88%

Precision

100%

Recall

99.74%

F1-Score

99.87%

XGBoost

Extreme Gradient Boosting

- Algoritmo di **gradient boosting** basato su alberi di decisione.
- Combina modelli deboli **in sequenza**, correggendo gli errori del precedente.
- Ogni nuovo albero cerca di correggere gli errori (residui) accumulati dagli alberi precedenti.
- Il modello finale è la **somma dei contributi** di tutti gli alberi.
- Il contributo di ogni albero è regolato dal **learning rate**: nuova previsione = previsione precedente + learning rate × correzione.
- Ad ogni iterazione il modello riduce progressivamente l'errore e migliora le prestazioni.

Accuracy

100%

Precision

100%

Recall

100%

F1-Score

100%

Risultati e Confronto Modelli

Metrica	Regressione Logistica	XGBoost
Accuracy	99.88%	100%
Precision	100%	100%
Recall	99.74%	100%
F1-Score	99.87%	100%

Entrambi i modelli mostrano performance eccellenti. L'XGBoost ha leggermente superato la Regressione Logistica in termini di **Accuracy**, **Recall** e **F1-Score**, indicando una perfetta capacità di identificare correttamente i funghi velenosi.

Sviluppi Futuri

Applicazione Nuovi Modelli

Random Forest, SVM, Neural Networks.

Miglioramenti dei modelli esistenti

Tuning degli Iperparametri e Cross-Validation.

Creazione nuove API

Funzionalità aggiuntive per l'utente, come l'aggiunta di nuovi record.